NOTE Chrome and Firefox's Network tab include a Copy as cURL option when you right-click a request. This allows you to copy a request from the browser as a curl command you can run from the command line. This makes curl even easier to use.

ISOLATION

Perhaps most importantly, curl provides isolation. When you make a request from JavaScript code, there are places where things can go wrong. The developer's JavaScript code may have a bug in it, or your assumptions of which headers the browser is adding may be different from what the browser is doing.

With curl, what you see on the command line is exactly what is sent to the server. If you have an error in your code, but the curl request works just fine, you know there is something wrong in your code. If you have an error in your code and the curl request fails, you know the issue is somewhere on your server.

7.5 Sending requests using test-cors.org

Full disclosure: I am the creator and maintainer of test-cors.org. I built the site because I found it difficult to find information on how CORS requests behave in actual browsers. The test-cors.org site offers four features for testing CORS requests:

- A curl-like interface to send CORS requests to a remote server. This can be useful for testing actual CORS requests from a browser against your server.
- A mechanism for configuring a server's response, then testing a request against this response. This can be useful if you want to learn how server and client settings interact.
- A mechanism to view events in an `XMLHttpRequest`'s lifecycle.
- A mechanism to get the JavaScript code for the request you're testing. These last two features are useful for client-side CORS users as well.

When you first visit test-cors.org, you're greeted with the page in figure 7.15.

The page is divided into three parts: the client panel, the server panel, and the results panel. Let's dig deeper into how these panels interact by exploring the various features of test-cors.org.

7.5.1 Sending requests to a remote server

The first way to use test-cors.org is to send cross-origin requests to a remote server. This is similar to how curl works, but it makes its request from the browser using an `XMLHttpRequest` object, rather than the command line. This can be helpful because it makes an HTTP request the same way you'd expect a client. For example, curl will let you add a Host header to the request. But the Host header can't be set from test-cors.org, because the browser controls this header.

To use test-cors.org to send a request to your sample server, begin by recreating the request to /api/posts in the client panel. This is a `GET` request that includes cookies and has the Timezone-Offset and Sample-Source headers set. Next, choose the Remote